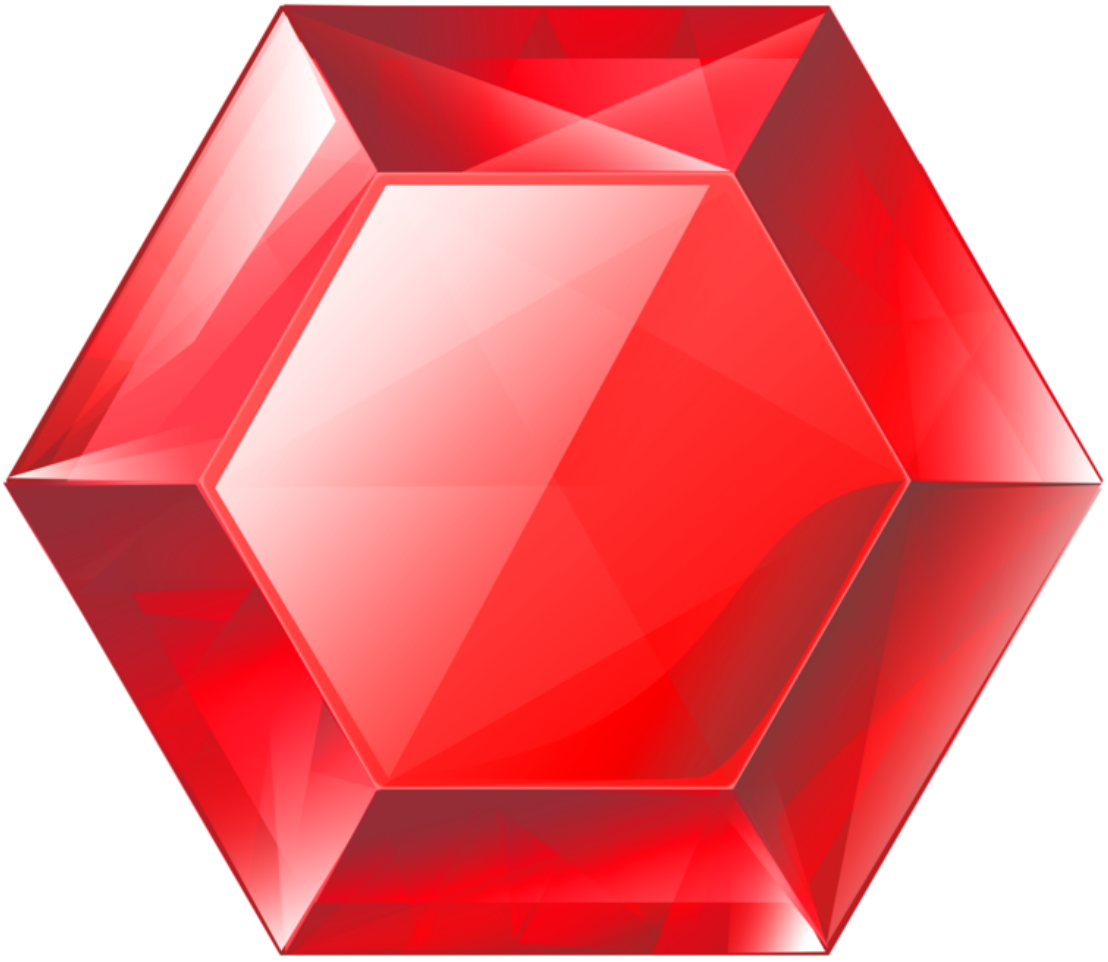
gemstone-rs Cookbook

Task-focused Rust recipes for GemStone/S



Generated from the Markdown sources in gemstone-rs/docs.

Contents

Cookbook

This cookbook is a collection of direct `gemstone-rs` recipes. Each recipe is small enough to copy into a Rust CLI, service, or maintenance tool.

Recipe 1: Open a Session and Evaluate Smalltalk

```
use gemstone_rs::{Config, Session};

let mut session = Session::login(Config::from_env()?)?;
println!("{:?}", session.eval("3 factorial")?);
```

Use this when you need the smallest live sanity check.

Recipe 2: Verify `3 + 4`

```
use gemstone_rs::{Config, Session, Value};

let mut session = Session::login(Config::from_env()?)?;
assert_eq!(session.eval("3 + 4")?, Value::SmallInt(7));
```

This is the test to keep in every live smoke lane.

Recipe 3: Write and Read `UserGlobals`

```
use gemstone_rs::{Config, Oop, Session};

let mut session = Session::login(Config::from_env()?)?;
let key = "GemStoneRsCookbook";
let value = session.new_string("stored from Rust")?;
session.global_put(key, value)?;

let stored = session.global_get(key)?;
println!("{}", session.fetch_string(stored)?);

session.global_put(key, Oop::NIL)?;
```

Recipe 4: Commit a Write Safely

```
session.transaction(|session| {
    let value = session.new_string("committed")?;
    session.global_put("GemStoneRsCommitted", value)
})?;
```

The closure commits only when it returns `Ok`.

Recipe 5: Abort on Error

```
use gemstone_rs::Error;

let result: gemstone_rs::Result<()> = session.transaction(|session| {
    let value = session.new_string("will abort")?;
    session.global_put("GemStoneRsAborted", value)?;
    Err(Error::IllegalOop {
        operation: "intentional abort",
    })
});

assert!(result.is_err());
```

Recipe 6: Convert Rust Values to OOPs

```
use gemstone_rs::Value;

let seven = session.value_to_oop(&Value::SmallInt(7))?;
let yes = session.bool_oop(true);
let nothing = session.nil_oop();
```

Recipe 7: Fetch a String

```
let string_oop = session.new_string("hello")?;
let text = session.fetch_string(string_oop)?;
assert_eq!(text, "hello");
```

Recipe 8: Send a Message and Keep the Raw OOP

```
let seven = session.smallint_oop(7);
let printed_oop = session.perform_oop(seven, "printString", &[])?;
println!("{}", session.fetch_string(printed_oop)?);
```

Recipe 9: Send a Message and Marshal the Result

```
let seven = session.smallint_oop(7);
let value = session.perform(seven, "class", &[])?;
println!("{value:?}");
```

Recipe 10: Retain a Long-Lived OOP

```
let text = session.new_string("retained")?;
let handle = session.retain_oop(text)?;
println!("retained OOP {}", handle.oop().raw());
handle.release()?;
```

The handle releases automatically on drop if `release()` is not called.

Recipe 10A: Store a Rust Payload Under BridgeRoot

```
use gemstone_rs::{BridgeValue, Config, Session};
use std::collections::BTreeMap;

let mut session = Session::login(Config::from_env()?)?;
let labels = BTreeMap::from([("source".to_string(), "cookbook".to_string())]);
let payload = BridgeValue::dictionary([
    ("name".to_string(), BridgeValue::from("Tariq")),
    ("amount".to_string(), BridgeValue::from(100_i64)),
    ("currency".to_string(), BridgeValue::from("GBP")),
    ("labels".to_string(), BridgeValue::from(labels)),
]);

let mut bridge_root = session.bridge_root()?;
bridge_root.put("MyTestDict", payload)?;
bridge_root.commit()?;
```

The default root is `UserGlobals at: #GemStoneRsBridgeRoot`.

Recipe 10B: Round-Trip a Typed BridgeRoot Map Field

```
use gemstone_rs::{
    BridgeDictionary, BridgeFieldWrite, BridgeKeyType, BridgeMapped, BridgeValue,
Config, Session,
};
use std::collections::BTreeMap;

#[derive(Debug, Eq, PartialEq)]
struct BookingDraft {
    name: String,
    labels: BTreeMap<String, String>,
}

impl BridgeMapped for BookingDraft {
    fn to_bridge_value(&self) -> BridgeValue {
        BridgeValue::dictionary([
            ("name".to_string(), BridgeValue::from(self.name.clone())),
            ("labels".to_string(),
BridgeFieldWrite::to_bridge_field_value(&self.labels)),
        ])
    }

    fn from_bridge_dictionary(dictionary: &mut BridgeDictionary<'_>) ->
gemstone_rs::Result<Self> {
        Ok(Self {
            name: dictionary.at_string("name")?,
            labels: dictionary.at_map("labels")?,
        })
    }
}

let mut session = Session::login(Config::from_env()?)?;
let draft = BookingDraft {
    name: "Tariq".to_string(),
    labels: BTreeMap::from([("source".to_string(), "cookbook".to_string())]),
};

let mut bridge_root = session.bridge_root()?;
bridge_root.put_mapped("CookbookBookingDraft", &draft)?;
let loaded: BookingDraft = bridge_root.get_mapped("CookbookBookingDraft")?;
assert_eq!(loaded.labels["source"], "cookbook");

bridge_root.put_string("CookbookStatus", "ready")?;
bridge_root.put_vec("CookbookTags", &["priority".to_string(), "demo".to_string()])?;
bridge_root.put_map("CookbookLabels", &draft.labels)?;
assert_eq!(bridge_root.get_string("CookbookStatus")?, "ready");
let tags: Vec<String> = bridge_root.get_vec("CookbookTags")?;
assert_eq!(tags, vec!["priority".to_string(), "demo".to_string()]);
let labels: BTreeMap<String, String> = bridge_root.get_map("CookbookLabels")?;
assert_eq!(labels["source"], "cookbook");
```

```
bridge_root.put_map_with_key_type("CookbookLabelsSymbol", BridgeKeyType::Symbol,
&draft.labels)?;
let symbol_labels: BTreeMap<String, String> =
    bridge_root.get_map_with_key_type("CookbookLabelsSymbol",
BridgeKeyType::Symbol)?;
assert_eq!(symbol_labels["source"], "cookbook");
```

Use map fields for string-keyed metadata or lookup tables. Use nested `BridgeMapped` structs when the related value has a stable domain shape.

Recipe 10C: Store a Symbol-Keyed Dictionary

Use `BTreeMap<String, T>` for string-keyed metadata. When the GemStone dictionary entries themselves need symbol keys, build a keyed dictionary explicitly:

```
use gemstone_rs::{BridgeKey, BridgeKeyType, BridgeValue, Config, Session};

let mut session = Session::login(Config::from_env()?)?;
let mut bridge_root = session.bridge_root()?;

let payload = BridgeValue::keyed_dictionary([
    (BridgeKey::symbol("status"), BridgeValue::from("ready")),
    (BridgeKey::symbol("amount"), BridgeValue::from(100_i64)),
    (BridgeKey::string("externalId"), BridgeValue::from("web-42")),
]);

bridge_root.put("CookbookSymbolDictionary", payload)?;

let mut dictionary = bridge_root.get_dictionary("CookbookSymbolDictionary")?;
assert_eq!(
    dictionary.at_string_with_key_type("status", BridgeKeyType::Symbol)?,
    "ready"
);
assert_eq!(
    dictionary.at_smallint_with_key_type("amount", BridgeKeyType::Symbol)?,
    100
);
assert_eq!(dictionary.at_string("externalId")?, "web-42");
```

`put_map_with_key_type` changes the key used to store the map in the containing dictionary. It does not change the entry keys inside the stored map. Use `BridgeValue::keyed_dictionary` for symbol-keyed entries inside the value.

Recipe 11: Browse Dictionaries

```
use gemstone_rs::browser::Browser;
```

```
let mut browser = Browser::new(&mut session);
for dictionary in browser.dictionaries()? {
    println!("{dictionary}");
}
```

Recipe 12: Browse a Class

```
use gemstone_rs::browser::{Browser, ALL_PROTOCOLS};

let mut browser = Browser::new(&mut session);
let protocols = browser.protocols("Object", false, "")?;
let methods = browser.methods("Object", ALL_PROTOCOLS, false, "")?;
let source = browser.source("Object", "printString", false, "")?;
```

Pass an empty dictionary to resolve through the active user's symbol list.

Recipe 13: Diagnose a New Machine

```
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --json
```

The first command checks environment and GCI loading, including whether `libgcirpc` came from explicit config, `GS_LIB_PATH`, `GS_LIB`, or `GEMSTONE/lib`, plus the exact path or directory searched. The second command logs in and verifies that `3 + 4` returns `7`. The JSON form is useful in CI or editor tooling.

Recipe 14: Inspect an OOP From the CLI

```
gemstone-rs inspect oop 20
```

This prints the raw OOP, class OOP, and `printString`.

Recipe 15: Inspect BridgeRoot From the CLI

```
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge get BookingDraft --symbol
gemstone-rs bridge inspect BookingDraft --symbol
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
```

```
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
```

Use this when object-mapping examples or generated mappings write payloads under `GemStoneRsBridgeRoot`. `bridge keys` is the quickest way to see what is currently stored before inspecting a specific payload. The `put-*` shortcuts and `bridge remove` are useful live-write smoke tests because they commit one explicit BridgeRoot change at a time. Use generic `bridge put --type ...` when you want the value type to be data-driven in a script.

Recipe 16: Preview Codegen

```
gemstone-rs codegen preview examples/codegen/gemstone-rs.codegen
```

Use preview before writing generated files.

Recipe 17: Check Generated Code in CI

```
gemstone-rs codegen check examples/codegen/gemstone-rs.codegen
```

The repository runs the same check through `make verify`.

Recipe 18: Generate Wrappers

```
gemstone-rs codegen generate examples/codegen/gemstone-rs.codegen
```

Generated files are meant to be checked in.

Recipe 19: Discover a Config From a Live Stone

```
gemstone-rs codegen discover examples/codegen/discovered.codegen Object
```

Use this to bootstrap a config, then edit the generated mapping down to the API you actually want.

Recipe 19A: Run a Minimal Rust HTTP Health Service

```
cargo run -p gemstone-rs --example http_service -- --routes
cargo run -p gemstone-rs --example http_service -- --port 3000
```

From a second terminal:

```
curl -i http://127.0.0.1:3000/
curl -i http://127.0.0.1:3000/health/local
curl -i http://127.0.0.1:3000/health/gemstone
```

The example uses only the Rust standard library. It proves the service shape and GemStone health-check flow without pulling web framework dependencies into the core crate. `gemstone-py` is still ahead for batteries-included FastAPI, Litestar, and Django adapters; `gemstone-rs` now has a direct Rust service smoke path plus packaged Axum and Actix adapter crates:

```
cargo run --manifest-path examples/axum-service/Cargo.toml -- --routes
cargo run --manifest-path examples/actix-service/Cargo.toml -- --routes
```

The Axum and Actix services now start a bounded `SessionWorkerPool`, then use `gemstone-rs-axum` or `gemstone-rs-actix` to expose `/`, `/health/local`, and `/health/gemstone` without copying handler code.

For local development, the framework examples use the non-failing health-pool startup path. The server can start before credentials are configured: `/` and `/health/local` stay available, while `/health/gemstone` returns a `503` JSON error until the stone is reachable. The route smoke script checks that behavior without needing a live stone:

```
python3 scripts/framework_route_smoke.py
scripts/live_smoke.sh --dry-run
scripts/live_smoke.sh
```

The adapters also return `x-gemstone-rs-adapter`, `x-gemstone-rs-route`, `x-gemstone-rs-request-id`, `x-gemstone-rs-request-method`, and `x-gemstone-rs-request-path` headers, plus `x-gemstone-rs-request-lifecycle: received,handled` and `x-gemstone-rs-request-duration-us`. The smoke script asserts those headers so a proxy, load balancer, or test can distinguish the framework adapter, route, request, lifecycle, and handler duration that produced the response. It also asserts `x-gemstone-rs-example-middleware: axum` or `x-gemstone-rs-example-middleware: actix`, `x-gemstone-rs-service`, `x-gemstone-rs-service-version`, `cache-control: no-store`, and `x-content-type-options: nosniff` from the checked services. That proves the packaged routes still compose with normal framework middleware and gives the examples a small

production-style cache/security header policy. In live mode the same script requires `/health/gemstone` to reach the stone and return `{"result":7}` for both adapters; the manual CI job runs that path with GemStone secrets.

Use `SessionWorker` when the application wants a reusable dedicated GemStone session lane instead of opening a new session inside each blocking route:

```
use gemstone_rs::{Config, SessionWorker, Value};

let worker = SessionWorker::start(Config::from_env()?)?;
assert_eq!(worker.eval("3 + 4")?, Value::SmallInt(7));
worker.shutdown()?;
# Ok::<(), gemstone_rs::Error>(())
```

Use `SessionWorkerPool` when the service should keep a fixed number of GemStone sessions warm and dispatch calls round-robin:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};

let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval("3 + 4")?, Value::SmallInt(7));
pool.shutdown()?;
# Ok::<(), gemstone_rs::Error>(())
```

The same pool exposes awaitable calls for async runtimes:

```
use gemstone_rs::{Config, SessionWorkerPool, Value};

let pool = SessionWorkerPool::start(Config::from_env()?, 2)?;
assert_eq!(pool.eval_async("3 + 4").await?, Value::SmallInt(7));
pool.shutdown()?;
# Ok::<(), gemstone_rs::Error>(())
```

The installed CLI can also scaffold this shape:

```
gemstone-rs examples scaffold session_worker_pool ./gemstone-rs-worker-pool
```

The dependency-free `gemstone_rs::web` helpers now use the same async worker facade in Axum and Actix health handlers, so framework routes no longer need to wrap GemStone health checks in framework-specific blocking helpers.

Recipe 20: Check the Python Native Adapter Contract

Use this when you are preparing `gemstone-py-native` to wrap the Rust core:

```
gemstone-rs py-native capabilities
gemstone-rs py-native capabilities --json
gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json
gemstone-rs py-native samples --json
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json
gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json
gemstone-rs py-native smoke --dry-run
gemstone-rs py-native migration --json
gemstone-rs py-native compatibility --json
gemstone-rs py-native check-compat examples/py-native/gemstone-rs.py-native-
compat.json
gemstone-rs py-native conformance --json
gemstone-rs py-native check-conformance examples/py-native/gemstone-rs.py-native-
conformance.json
gemstone-rs py-native handoff --json
gemstone-rs py-native check-handoff examples/py-native/gemstone-rs.py-native-
handoff.json
gemstone-rs py-native publish-receipt --json
gemstone-rs py-native check-publish-receipt examples/py-native/gemstone-rs.py-native-
publish-receipt.json
gemstone-rs py-native check-all
gemstone-rs py-native check-all --json
cargo run -p gemstone-rs --example python_native_adapter -- --dry-run
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
```

The CLI command prints the contract version, threading rule, supported operations, value kinds, error kinds, and OOP constants. The fixture checks compare the checked-in capability, value/error sample, and dry-run smoke JSON against the shared core renderer. `py-native samples --json` gives Python wrapper CI concrete payloads for `nil`, booleans, small integers, characters, strings, symbols, OOPs, and structured errors. The smoke command checks capabilities, OOP constants, value conversion, config error mapping, and structured error mapping without a live stone when `--dry-run` is passed. The `py-native migration --json` report turns the shared-core path into a concrete checklist for `gemstone-py-native`: wrap `PyNativeSession`, preserve the Python API, keep live backend smoke green, and verify published wheels from TestPyPI/PyPI. `py-native compatibility --json` adds a method-by-method map for the generated Python shim, including `OopHandle` return points and typed helper methods. The `py-native conformance --json` report adds the integration target: generated module functions, raw `NativeSession` methods, compatibility shim methods, fixture files, and scaffold files. `py-native handoff --json` adds the final downstream manifest tying the contract, samples, smoke, migration, compatibility, conformance, and acceptance checks together. `py-native publish-receipt --json` records the verified TestPyPI and PyPI workflow runs, install commands, and package checks for the Rust-backed `gemstone-py-native` wheel release. The `py-native check-all` gate validates all

checked-in py-native fixtures together, which is the shortest command to put in downstream `gemstone-py-native` CI before publishing native wheels. The PyO3 scaffold writes a minimal `gemstone_py_native` extension module with `capabilities_json`, `samples_json`, `smoke_dry_run_json`, `migration_json`, `compatibility_json`, `conformance_json`, `handoff_json`, and an unsendable `NativeSession` wrapper over eval, execute, resolve, value-to-OOP conversion, perform, strings, symbols, globals, export-set retention, and transactions. The wrapper also exposes `eval_json` and `perform_json` so the Python package layer can decode the stable `PyNativeValue` JSON shape instead of inferring native values itself. It also writes `python/gemstone_py_native_compat.py`, a Python package-layer shim with `NativeCompatibilitySession`, `OopHandle`, and a `compatibility_report()` that documents the return policy: object identity returns handles, raw native OOPs stay below the package boundary, value-returning methods become dictionaries, and typed helpers are opt-in. The live example logs in, evaluates `3 + 4`, performs `printString`, and round trips a `UserGlobals` string through `PyNativeSession`:

```
gemstone-rs py-native smoke
cargo run -p gemstone-rs --example python_native_adapter
```

The important point is architectural: Python should wrap `gemstone_rs::py_native`; it should not duplicate dynamic GCI loading or raw session calls.

Recipe 21: Run the Local Explorer

```
gemstone-rs-explorer --port 8787
```

Open:

```
http://127.0.0.1:8787/
```

The explorer starts read-only and loopback-only.

Recipe 22: Add a Local Explorer Auth Token

```
export GEMSTONE_RS_EXPLORER_TOKEN='replace-with-a-local-random-token'
gemstone-rs-explorer --port 8787 --auth-token-env GEMSTONE_RS_EXPLORER_TOKEN
curl -s 'http://127.0.0.1:8787/api/config?token=replace-with-a-local-random-token'
```

VS Code users can run `GemStone RS: Generate Explorer Auth Token`; the workbench stores the token in `gemstoneRs.explorerAuthToken`, launches the explorer with `--auth-token-env GEMSTONE_RS_EXPLORER_TOKEN`, and opens token-aware browser/webview URLs.

Recipe 23: Enable Explorer Eval Explicitly

```
gemstone-rs-explorer --port 8787 --allow-eval
curl -s 'http://127.0.0.1:8787/api/eval?source=3%20%2B%204'
```

Use this locally only.

Recipe 24: Use the VS Code Workbench With a Checkout

```
{
  "gemstoneRs.checkoutPath": "/path/to/gemstone-rs",
  "gemstoneRs.useCargo": true,
  "gemstoneRs.codegenConfig": "examples/codegen/gemstone-rs.codegen"
}
```

Then open the GemStone RS sidebar and use the Dictionaries, Codegen Config, and Explorer trees.

Recipe 25: Verify Published Artifacts

```
scripts/publish_verify.sh 0.2.2
```

The script checks crates.io versions, installs the CLI and explorer, runs `--help`, and checks the Marketplace version.

> This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.